



Multilevel Business Process Modeling: Motivation, Approach, Design Issues, and Applications

Christoph Schütz*
Johannes Kepler University
Linz, Austria
schuetz@dke.uni-
linz.ac.at

Michael Schrefl
Johannes Kepler University
Linz, Austria
schrefl@dke.uni-
linz.ac.at

Lois M. L. Delcambre†
Portland State University
Portland, Oregon, USA
lmd@cs.pdx.edu

ABSTRACT

Conceptual models organize the data as well as the processes from a business domain using a human-readable, yet formal representation language. For static data models, various modeling approaches have been proposed to represent complex multilevel abstraction hierarchies. For process models, though, current modeling approaches do not provide a flexible and powerful formalism for representing complex multilevel abstraction hierarchies. In this paper, we motivate the need for multilevel business process models, present our initial approach, and sketch future research in this area.

Categories and Subject Descriptors

H.4.1 [Information Systems Applications]: Office Automation—*Workflow management*; D.2.2 [Software Engineering]: Design Tools and Techniques—*Object-oriented design methods*

General Terms

Design

Keywords

Conceptual Modeling, Metamodeling, Multilevel Abstraction, Artifact-centric Process Modeling

1. MOTIVATION

Typically, a company is a hierarchical organization comprising strategic, administrative, and operational units. In many cases, the data within a company are structured in multilevel abstraction hierarchies, reflecting to some extent the company's organization and the different information

*This work was supported by a Marshall Plan Scholarship granted by the Austrian Marshall Plan Foundation for a research stay at Portland State University.

†This work was supported in part by the United States National Science Foundation, Grant Number 0840668.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

PIKM'12, November 2, 2012, Maui, Hawaii, USA.

Copyright 2012 ACM 978-1-4503-1719-1/12/11 ...\$10.00.

needs of the various units. In order to better represent hierarchical data, conceptual modeling approaches with expressive mechanisms for multilevel abstraction have been proposed.

Aggregation is a common modeling construct where lower-level, more concrete data objects are collected into higher-level, more abstract aggregates. These aggregates may have a distinct set of attributes and provide metadata for the lower-level data objects. For example, an insurance claim has an amount which is constrained by the coverage indicated in the contract that the claim belongs to. These aggregates may again be collected into aggregates. For example, an insurance contract belongs to a particular insurance category. An insurance category is an aggregate consisting of several contracts.

Generalization is a modeling construct where the common features of different types of data objects are encapsulated in a common supertype, the generalization. The subtypes are specializations of this supertype. For example, insurance claims are filed for a particular amount of money. Claims under a health insurance contract are filed for a particular medical treatment in addition to specifying the amount of money. A supertype for all insurance claims defines the amount of money as a common attribute. The type for health insurance claims specializes the common supertype by adding an attribute for the medical treatment.

Some modeling approaches go beyond generalization and aggregation relationships to represent complex multilevel abstraction hierarchies. Popular approaches with support for multilevel (meta-)modeling are powertypes [16, 5], deep instantiation [1], and materialization [17, 3]. The instances of a powertype are subtypes of another object type, thereby providing metamodeling capabilities [16, p. 28]. Another powertype-based approach [5] abandons traditional two-level instantiation with class and object, using the notion of the “clabject” instead. This “class/object duality” is also an important feature of deep instantiation [1] that allows arbitrary-depth instantiation hierarchies where data objects can instantiate (certain aspects of) other data objects which instantiate other data objects, and so on. Materialization, on the other hand, blurs the boundaries between the aggregation and instantiation relationships.

The multilevel object (m-object) introduced by Neumayr et al. [11] is an expressive and flexible approach to modeling multilevel abstraction hierarchies [13] that combines elements from powertypes, deep instantiation, and materialization. An m-object has several abstraction levels ordered from the most abstract to the most concrete. Each

of these levels has an associated class. These classes are connected by aggregation relationships, constituting an aggregation hierarchy. The introduction of the concretization relationship for m-objects allows for the specialization of entire sub-hierarchies.

Early conceptual modeling approaches focused merely on the static structure of data while neglecting the dynamic aspects. But, the processes dealing with these data are equally important for running a business. This led to the emergence of data-centric business process modeling approaches, for example, object/behavior diagrams [9] and state machine and activity diagrams in UML. More recently, business artifacts [14, 6] have been successfully introduced in both theory and practice. The underlying principle of such data-centric approaches to business process modeling remains unchanged: encapsulate in a single object both data and process information.

In business process modeling, hierarchical modeling commonly refers to the description of the same process at different levels of granularity in order to hide unnecessary details from the user. For example, an insurance claim is assessed before it can be paid. But, the assessment phase can also be viewed in more detail, as consisting of an investigation and a decision phase. Many languages allow for the representation of such hierarchical models. UML state machines support sub-states and sub-machines [15]. More recently, Smirnov et al. [24], for example, allow the same process to be described at different levels of detail. More prominently, the guard-stage-milestone modeling language introduces such hierarchies for business artifacts [7].

Previous work has identified flexibility [19, 18] and variability [29] as important issues in business process modeling. For example, a health insurance claim requires that a medical examination takes place during the assessment phase, unlike other insurance categories. Case handling [27] provides the actors carrying out process tasks with a great deal of flexibility. Due to the need for flexibility in business process modeling, rules that govern specialization and change have also been proposed. Behavior-consistent specialization draws from object-oriented data modeling principles and defines precise rules for extending and refining business processes [21, 2]. Closely related to the matter of flexibility is the demand for supporting dynamic changes of models during the execution of a process [20, 28].

In conceptual process models, processes running at different hierarchical levels within a company are often viewed separately. But, in reality, these processes are interdependent. The representation of complex hierarchies in business process modeling could lead to a better alignment of the processes at different hierarchical levels within a company. Existing approaches towards conceptual business process modeling lack modeling constructs combining aggregation and generalization of processes for the representation of complex abstraction hierarchies. M-objects offer advanced multilevel modeling but focus only on the static aspects of data. We propose to extend m-objects with life cycle models to create a multilevel business process modeling approach.

The remainder of this paper is organized as follows. In Section 2, we present the modeling approach from our previous paper [23] and illustrate this approach with an example. In Section 3, we discuss the most important design issues to be solved in realizing our approach to business process modeling as well as possible applications.

2. APPROACH

We propose the multilevel business artifact (MBA) as an extension of the m-object for business process modeling. An MBA, as introduced by Schütz et al. [23], is an m-object which associates with each level of abstraction a single life cycle model that defines the legal execution order of the methods of the class associated with the respective level. Thus, in the spirit of business artifacts and m-objects, an MBA encapsulates in a single object information about the static and dynamic aspects of multiple levels of abstraction.

We use UML state machines consisting of states and transitions for modeling life cycles. Methods are associated with transitions. The method may only be invoked if its object is in the right state. The invocation of a method triggers the corresponding transition, causing a change in the current state of the object whose method has been invoked. UML state machines allow parallel transition paths and sub-states, and provide advanced modeling primitives, for example, pre- and post-conditions for further restricting the triggering of transitions. Other models could be used, such as Petri nets [26] or the guard-stage-milestone approach.

Figure 1 shows an example. MBA *Insurance*, for example, has levels *range*, *category*, *contract*, and *claim*. Level *range* is the most abstract, level *claim* is the most concrete level. Each box on the left-hand side represents a class associated with one of these levels. The two compartments within a box contain attribute and method definitions, respectively. At level *range*, for example, MBA *Insurance* defines attribute *description* as well as methods *create* and *addCategory*.

On the right-hand side of Figure 1, a state machine diagram represents the life cycle model for the corresponding class for each level. MBA *Insurance* at level *contract*, for example, has two states: *Negotiating* and *In Effect*. After creation, MBA *Insurance* is in the *Negotiating* state where only methods *setCoverage* and *conclude* may be invoked. The invocation of method *conclude* triggers a change to state *In Effect* where only method *fileClaim* may be invoked. A pre-condition specifies that method *fileClaim* may not be invoked if the value of parameter *amount* is greater than the value of the object's *coverage* attribute. Pre- and post-conditions are specified using the Object Constraint Language (OCL).

An important feature of MBAs (and m-objects) is their class/object duality. An MBA is an instance of its top-level class but contains only schema information for the more concrete levels. Likewise, an MBA is in a particular state of its top-level life cycle model. For example, in Figure 1, MBA *Insurance* is an instance of the class associated with level *range*, assigning a value to attribute *description* and being in the *Analyzing* state.

Multilevel concretization allows for the specialization of the data and process models of entire sub-hierarchies. For example, in Figure 1, MBA *Health* is a concretization of *Insurance*. MBA *Health* is an MBA at the *category* level and inherits from *Insurance* every level from *category* downwards. The attributes and methods defined by *Health* are added to those defined by *Insurance*. MBA *Health* also specializes the state machines inherited from its parent. In Figure 1, gray color marks the states and transitions inherited from *Insurance*. In addition, at the *contract* level, MBA *Health* has a state *Amending*, parallel to state *In Effect*, and adds a transition to the *Negotiating* state as well as modifying the pre-condition of an inherited transition. At the

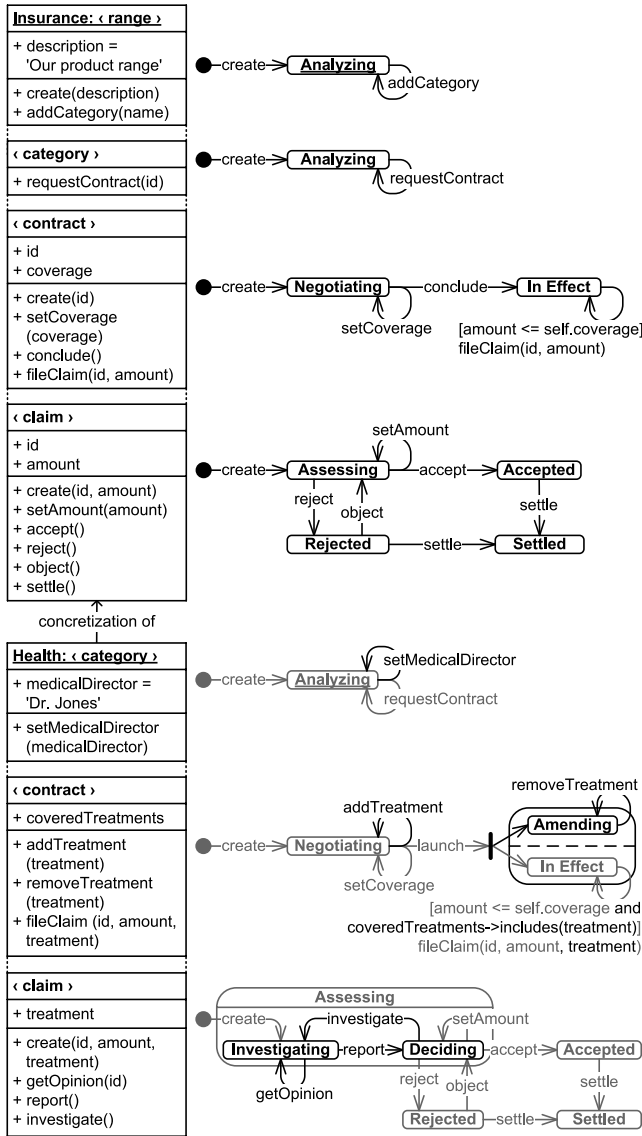


Figure 1: An MBA and one of its concretizations for the management of insurance data

claim level, MBA *Health* refines the *Assessing* state. We refer to Stumptner and Schrefl [25] for a formal definition and a more detailed presentation of behavior-consistent specialization in UML.

3. DESIGN ISSUES & APPLICATIONS

A modeler creating a concretization of some other MBA specializes the schema. Therefore, concretization provides modelers with flexibility. Multilevel concretization is not an atemporal operation but is itself a meta-process. By referencing reflective methods in the life cycle models of MBAs, the flexibility of modelers may be deliberately restricted [23]. Schema specialization could be restricted to certain phases in the life cycle of an object. For example, states and transitions might only be added to a life cycle model while in the *Developing* state. Pre- and post-conditions could be used to further restrict the use of reflective functions. Pre- and post-

conditions could also be used to restrict the introduction of new levels. This approach towards flexibility is similar to the idea of the two-tier artifact-centric framework proposed by Liu et al. [10] with process design entities and business entities, but on multiple levels of abstraction and incorporating behavior-consistent specialization.

The different levels of a single MBA do not exist in isolation but are interdependent, interact with each other, and need coordination. For example, in Figure 1, the *range* level of MBA *Insurance* determines the creation of objects at the *category* level. After the execution of the *addCategory* method, an additional category with the given name exists. Furthermore, higher-level objects constrain the values of lower-level objects. For example, attribute *coverage* at level *contract* of MBA *Insurance* constrains the values of attribute *amount* at the *claim* level. Schütz et al. [23] propose the use of pre- and post-conditions in UML state machines to model interaction and coordination between levels.

Besides the coordination of processes at different levels within the same MBA, in many practical situations, the processes associated with different MBAs also need coordination. For data models, multilevel relationships (m-relationships) have been proposed for connecting different m-objects at multiple levels of abstraction [11]. In process modeling, m-relationships between MBAs would represent the passing of messages for coordination purposes. The implications on behavior-consistent specialization will have to be addressed. Future work could build on the findings of Yongchareon et al. [30] about specialization of synchronization dependencies in artifact-centric process models.

A process model must also capture the actors that carry out the tasks. Even though MBAs, in general, represent business objects, modeling actors as MBAs is not a paradox. Each actor has a distinct set of attributes and many actors are in a hierarchical relationship with each other, for example, employees and departments. Thus, from a database perspective, these actors are data objects which could also be linked to other data objects. Furthermore, MBAs could be extended with activity diagrams, each corresponding to a specific process as seen from the actor's point of view and referencing the methods of the processed MBAs used to complete the tasks. This approach would be similar to subject-oriented process modeling, where an actor's tasks are defined in terms of a business object's methods [4]. These activity diagrams would need coordination across different abstraction levels. An alternative to modeling actors as MBAs is the integration of MBAs in existing modeling approaches.

In order to support modelers with applying the MBA approach, tool support should be devised. As a first step, a prototype for the management of MBAs could be realized based on the MBA metamodel in UML [23]. The ConceptBase [8] deductive database system provides a flexible data model that can be used to represent the MBA metamodel. This implementation could serve as a basis for visual modeling tools and as a central repository for MBAs which is also used to check consistency. In a second step, the integration of MBAs in existing execution environments could be undertaken. The central repository might still be a core element and complement other environments. Application code could be executed locally while validity checks are done in the central repository. But, application code could also be executed in the central repository, reducing local application code to establishing the connection to the server.

MBAs could also be applied to data warehouse modeling for performance analysis of processes. An important issue in data warehouse modeling is the representation of heterogeneities. In order to solve this issue, Neumayr et al. [12] introduce hetero-homogeneous dimensions and cubes modeled with m-objects which feature a globally homogeneous schema but allow for the introduction of heterogeneities in well-defined partitions of the data. The hetero-homogeneous approach to data warehouse modeling could be adapted for business process analysis. Different processes could be associated with different measures and aggregated along different aggregation levels. The proof-of-concept prototype system for the management of hetero-homogeneous data warehouses [22] could be extended in order to support performance measurement for business processes, leading to the development of a hetero-homogeneous process warehouse.

4. REFERENCES

- [1] C. Atkinson and T. Kühne. The essence of multilevel metamodeling. In *Proc. UML '01*, volume 2185 of *LNCS*, pages 19–33. 2001.
- [2] T. Basten and W. van der Aalst. Inheritance of behavior. *J. of Logic and Algebraic Programming*, 47(2):47–145, 2001.
- [3] M. Dahchour, A. Pirotte, and E. Zimányi. Materialization and its metaclass implementation. *IEEE TKDE*, 14(5):1078–1094, 2002.
- [4] A. Fleischmann and C. Stary. Whom to talk to? A stakeholder perspective on business process development. *Universal Access in the Information Society*, 11:125–150, 2012.
- [5] C. Gonzalez-Perez and B. Henderson-Sellers. A powertype-based metamodelling framework. *Software & System Modeling*, 5(1):72–90, 2006.
- [6] R. Hull. Artifact-centric business process models: Brief survey of research results and challenges. In *Proc. OTM '08*, volume 5332 of *LNCS*, pages 1152–1163. 2008.
- [7] R. Hull, E. Damaggio, R. De Masellis, F. Fournier, M. Gupta, F. T. Heath, S. Hobson, M. H. Linehan, S. Maradugu, A. Nigam, P. N. Sukaviriya, and R. Vaculín. Business artifacts with guard-stage-milestone lifecycles: Managing artifact interactions with conditions and events. In *Proc. DEBS '11*, pages 51–62, 2011.
- [8] M. Jeusfeld, M. Jarke, and J. Mylopoulos. *Metamodeling for method engineering*. MIT Press, 2009.
- [9] G. Kappel and M. Schrefl. Object/behavior diagrams. In *Proc. ICDE '91*, pages 530–539, 1991.
- [10] R. Liu, F. Y. Wu, F. Pinel, and Z. Shan. A two-tier data-centric framework for flexible business process management. In *Proc. AMCIS '12*, 2012.
- [11] B. Neumayr, K. Grün, and M. Schrefl. Multi-level domain modeling with m-objects and m-relationships. In *Proc. APCCM '09*, pages 107–116, 2009.
- [12] B. Neumayr, M. Schrefl, and B. Thalheim. Hetero-homogeneous hierarchies in data warehouses. In *Proc. APCCM '10*, pages 61–70, 2010.
- [13] B. Neumayr, M. Schrefl, and B. Thalheim. Modeling techniques for multi-level abstraction. In *The Evolution of Conceptual Modeling*, volume 6520 of *LNCS*, pages 68–92. 2011.
- [14] A. Nigam and N. S. Caswell. Business artifacts: An approach to operational specification. *IBM Systems Journal*, 42(3):428–445, 2003.
- [15] Object Management Group. *OMG Unified Modeling Language (OMG UML), Superstructure, version 2.4.1*, 2011.
- [16] J. Odell. *Advanced object-oriented analysis and design using UML*, chapter Power types, pages 23–32. Cambridge University Press, 1998.
- [17] A. Pirotte, E. Zimányi, D. Massart, and T. Yakusheva. Materialization: A powerful and ubiquitous abstraction pattern. In *Proc. VLDB '94*, pages 630–641, 1994.
- [18] G. Redding, M. Dumas, A. ter Hofstede, and A. Iordachescu. A flexible, object-centric approach for business process modelling. *Service Oriented Computing and Applications*, 4(3):191–201, 2010.
- [19] M. Reichert, S. Rinderle-Ma, and P. Dadam. Flexibility in process-aware information systems. In *Proc. ToPNoC II*, volume 5460 of *LNCS*, pages 115–135. 2009.
- [20] S. Rinderle, M. Reichert, and P. Dadam. Correctness criteria for dynamic changes in workflow systems – a survey. *DKE*, 50(1):9–34, 2004.
- [21] M. Schrefl and M. Stumptner. Behavior-consistent specialization of object life cycles. *ACM ToSEM*, 11(1):92–148, 2002.
- [22] C. Schütz. Extending data warehouses with hetero-homogeneous dimension hierarchies and cubes: A proof-of-concept prototype in Oracle. Master's thesis, JKU, Linz, Austria, 2010.
- [23] C. Schütz, L. Delcambre, and M. Schrefl. Multilevel business artifacts. In *Proc. DAB '12*, 2012. To appear. <https://sites.google.com/site/dabworkshop2012/>.
- [24] S. Smirnov, H. Reijers, and M. Weske. A semantic approach for business process model abstraction. In *Proc. CAiSE '11*, volume 6741 of *LNCS*, pages 497–511. 2011.
- [25] M. Stumptner and M. Schrefl. Behavior consistent inheritance in UML. In *Proc. ER 2000*, volume 1920 of *LNCS*, pages 451–530. 2000.
- [26] W. van der Aalst. The application of Petri Nets to workflow management. *J. of Circuits, Systems, and Computers*, 8(1):21–66, 1998.
- [27] W. van der Aalst, M. Weske, and D. Grünbauer. Case handling: A new paradigm for business process support. *DKE*, 53(2):129–162, 2005.
- [28] B. Weber, M. Reichert, and S. Rinderle-Ma. Change patterns and change support features - enhancing flexibility in process-aware information systems. *DKE*, 66(3):438–466, 2008.
- [29] M. Weidlich, J. Mendling, and M. Weske. A foundational approach for managing process variability. In *Proc. CAiSE '11*, volume 6741 of *LNCS*, pages 267–282. 2011.
- [30] S. Yongchareon, C. Liu, and X. Zhao. A framework for behavior-consistent specialization of artifact-centric business processes. In *Proc. BPM '12*, volume 7481 of *LNCS*, pages 285–301. 2012.